ANTHROP\C

Appendix to "Agentic Coding and Persistent Returns to Expertise"

June 2026

Sample

Our sample covers interactive Claude Code sessions between October 2025 and April 2026 (inclusive). This includes sessions conducted through the Claude Code command-line interface, the Claude Code desktop app, and claude.ai. This excludes Claude Code usage via third party integrated developer environments as well as via software development kits. Sessions with zero human turns are excluded—these excluded sessions are non-interactive calls in "headless" mode i.e., via claude -p "<prompt>".

The analysis covers 398,198 sessions uniformly randomly sampled from Claude Code traffic. These sessions come from 234,751 users. 73,066 users appear more than once in the sample, 6,382 users appear more than 5 times and 1,413 users appear more than 10 times. We exclude internal Anthropic usage.

All classifications ran through our privacy-preserving analysis infrastructure. Researchers never read individual sessions or transcripts, and every number in this report is an aggregate over a cell of sessions that is suppressed unless it contains a minimum number of distinct users.

For each classifier call, the model is shown the session transcript (human, assistant, and system turns) with each turn middle-truncated at 5,000 characters and the transcript as a whole truncated at 25,000 characters. Almost every classifier below is a single call to Claude Sonnet 4.6 at temperature 0.2 that must answer with exactly one of the listed options—the exception is the estimated task value classifier which uses Claude Haiku 4.5 for an initial summarizer prompt and Opus 4.7 for an estimation prompt.

Alongside the classifiers, we use telemetry recorded automatically for every session. The telemetry we use here includes the number of human prompts ("turns"), model calls (API requests), successful tool calls, output tokens, lines of code added and removed and the age of the user's account (days since signup with Anthropic, not with Claude Code specifically).

Classifier Prompts

Work Mode

This classifier categorizes each session into one of nine work modes that best describes what the session is trying to accomplish.

```
Classify the WORK MODE of this Claude Code session. Pick EXACTLY ONE of:
Plan, Build, Fix, Understand, Test, Operate, Analyze, Orchestrate, or
Communicate. Each label is a verb describing what the user is doing.

There is NO 'Unclear' option — pick the BEST fit from the nine, even when
evidence is thin. If the session is short or ambiguous, pick the verb that
best fits the small amount of evidence rather than abstaining.

**Pick by DOMINANT activity, measured by volume of meaningful work, NOT by
the user's stated intent in turn 1.** If a session opens with planning but
then spends most turns writing code, that's Build, not Plan. The triggering
intent doesn't decide the label — the bulk of the work does.

Key distinctions:
**Build vs Fix**: Build adds, evolves, refactors, optimizes WORKING code.
Fix is reactive — the trigger is something broken.
**Build vs Analyze**: Build = code is the deliverable for ongoing use.
Analyze = code is INSTRUMENTAL; the artifact is an insight, chart, or
model.
**Build vs Test**: writing tests is Test. Fixing app code revealed broken
by tests is Fix. Building test infrastructure itself is Build.
**Build vs Operate**: shipped code is Build. Deploy / config / CI / env /
monitoring is Operate.
**Plan vs Understand**: Plan looks forward (designing). Understand looks at
what exists.
**Understand vs Analyze**: Understand is about code / systems / concepts.
Analyze is about data.
**Understand vs Fix**: investigation with no fix = Understand.
Investigation that resolves into a code change to make broken behavior
correct = Fix.
**Orchestrate**: meta-AI work. Apps that happen to call an LLM are Build.
**Communicate**: deliverable is human-readable words / slides. Pairs with
codebase_context = 'Not in a codebase'.
```

The options are below:
- Plan: Pre-implementation strategy. The user is figuring out HOW to do something before doing it: architecture, specs, design docs, tradeoff analysis, sequencing, scoping. The deliverable is a plan / spec / decision, NOT working code. If the session starts with planning but then transitions into building, classify by the dominant activity.
- Build: Write, refactor, migrate, or extend code. The user is producing or evolving code that is ITSELF the deliverable — a service, library, CLI, script, endpoint, component, ETL pipeline, configuration consumed by software. Subsumes new code, feature additions, refactors, optimizations, dependency updates, and migrations. Use Build whenever the primary output of the session is delivered or maintained code.
- Fix: Reactive fault diagnosis and bug fix. The session is triggered by code or behavior that is BROKEN — produces wrong output, errors, crashes, fails tests, returns the wrong thing, regresses an earlier behavior. The work is finding the cause and making it right. NOT this: code that works correctly but is slow / suboptimal / ugly (that's Build). NOT this: writing tests for working code (that's Test).
- Understand: Explore, investigate, root-cause-for-comprehension, audit-for-understanding. Primary output is KNOWLEDGE about a system, code, or concept — not a code change. Code review for understanding, tracing how a flow works, learning a new framework, investigating WHY something behaves a certain way without intent to change it, onboarding to a codebase. NOT this: investigation that resolves into a fix (that's Fix). NOT this: data analysis (that's Analyze). Hallmark: knowledge in the user's head at the end, not code in the repo.
- Test: Test, code review, QA, security audit. Deliverable is VERIFICATION of correctness — writing new tests, fixing or extending the test suite, security or vulnerability review, QA / acceptance checking, formal-method-style verification. Includes test infra, fixtures, mocks, coverage tooling. NOT this: fixing application code revealed broken by tests (that's Fix). NOT this: code review aimed primarily at learning the codebase (that's Understand).
- Operate: Deploy, configure, monitor, on-call, CI/CD, env / dependency setup. The session works on the SCAFFOLDING around code rather than the code itself: Dockerfiles, k8s manifests, Terraform, GitHub Actions / CI workflows, deploy scripts, env-var management, package install, on-call paging, log triage, monitoring dashboards, infra provisioning, rolling out a release. NOT this: writing infra-as-code as part of building a new infra TOOL (that's Build). Hallmark: operational / lifecycle work.
- Analyze: Data analysis, research coding, analytics. Primary output is analytical artifacts — numbers, tables, plots, reports, models — where any code is INSTRUMENTAL (one-shot scripts, notebooks, exploratory queries)

```
rather than the deliverable. Examples: computing summary statistics,
building plots, running a regression, exploring a dataset, training a
one-off ML model for analysis, writing a SQL query for a report. NOT this:
production ETL pipelines or recurring data services (that's Build). NOT
this: code review of analysis code (that's Test or Understand). Hallmark:
the artifact at the end is an INSIGHT, not running software.
- Orchestrate: Orchestrate Claude or other AI agents — meta-work where
the deliverable is an AI artifact. Prompt engineering, designing agent
workflows, building eval harnesses for LLMs, tuning agent behavior,
debugging an agent, drafting agent system prompts, writing classifiers
powered by Claude, building tool-use loops, constructing few-shot or many-
shot examples. NOT this: writing application code that happens to call an
LLM as a feature (that's Build with an LLM/Agents domain). Hallmark: the
session's primary intellectual work is on the AI system itself.
- Communicate: Non-code knowledge work for a HUMAN AUDIENCE. The user is
using Claude Code as a general assistant to produce, refine, or work out
PROSE / SLIDES / MESSAGES / NOTES intended for human readers: business
writing (memos, emails, briefs, slides, reports), drafting / editing
essays or narratives, synthesizing external research into a writeup,
personal knowledge management (notes, summaries, journaling, organizing
thinking), planning a meeting / trip / decision in prose, brainstorming
captured as text, conversational math or finance done for a writeup.
Hallmark: deliverable is words / slides / human-readable artifacts, NOT
code, NOT analytical numbers, NOT AI-system tuning.
```

User expertise

This classifier measures the strength of the user's expertise exhibited in a session, on a five-point scale.

```
Classify the user's apparent EXPERTISE in the specific domain/task they
are attempting in this session. Pick EXACTLY ONE of these labels: 1, 2, 3,
4, 5, or Unclear, where 1 = Novice, 2 = Beginner, 3 = Intermediate, 4 =
Advanced, 5 = Expert, and Unclear when evidence is too thin.

This measures domain-specific familiarity with the WORK being done —
command of the terminology, structures, and conventions of whatever they
are doing in this session. Someone can be a senior software engineer but
a beginner at Rust, SOC coding, or differential privacy — rate them on
the task AT HAND, not their career. Rate domain expertise, NOT general
intelligence, NOT Claude's performance, and NOT task difficulty.
```

Weigh these three signals TOGETHER — they are co-equal, not ranked:

SETUP SPECIFICITY — does the user frame the problem using named entities and constraints that require domain knowledge to even reach for? ‘analyze this data’ vs. ‘compute the IRR over the cohort tagged _2024H2’ sit at different ends. Naming files or paths that are visible on screen is NOT domain knowledge — anyone using Claude Code does that. Weight things the user could only say if they already knew the domain.

VERIFICATION TYPE — what kind of verification does the user ask for? Generic asks (‘please double-check’, ‘are you sure?’, ‘verify your work’) are EPISTEMIC HUMILITY, not expertise — a careful novice does this. Targeted asks (‘show me how you set the config for X’, ‘did you actually call commit()?’, ‘what is the cardinality of that join?’) require knowing WHAT to check, which is expertise. Rate the TYPE, not the presence.

DIRECTION OF CORRECTION — who corrects whom on domain matters? If Claude has to correct the user’s terminology, mental model, or approach (‘actually, X doesn’t work that way — you probably want Y’), that pulls the rating DOWN toward 1-2. If the user catches Claude’s domain mistakes or steers away from a wrong approach Claude proposed, that pulls UP toward 4-5. If neither side corrects the other on domain matters, this signal is neutral.

The options are below:
- 1: Novice. Framing is generic or imprecise; does not use domain-specific names for the things being worked on. Verification asks are absent or entirely generic (‘please double-check’). Does not notice when Claude produces wrong output. Claude has to supply or correct basic domain concepts the user did not bring. May be articulate in prose, but the words are in service of asking for help, not directing a piece of domain work.
- 2: Beginner. Framing uses some correct terminology, but loosely or with occasional mis-use. Verification asks are mostly generic with at most occasional targeted ones. Pushes back on obviously wrong outputs but misses subtle ones. Claude corrects or reframes the user’s approach or terminology at least once. Specifies WHAT they want at a high level but not the specific shape, constraints, or invariants. A fluent technologist working OUTSIDE their domain — articulate, names the files in front of them, states goals clearly, but lacks the domain-specific mental model — lands here, not at 3 or 4.
- 3: Intermediate. Framing is precise at a directive level — names the files, the outputs, the categories, the libraries — but does not engage with methodology or tradeoffs. Mix of generic and targeted verification. Catches meaningful mistakes but does not routinely invoke specific

```
technical reasoning when correcting. Neither side is consistently
correcting the other on domain matters. A domain expert delegating a
routine task they do not need to think hard about can also land here.
- 4: Advanced. Framing shows structural domain knowledge in at least one
way that is NOT readable off the screen: names a specific edge case, non-
obvious constraint or invariant, version-specific behavior, or known
failure mode; describes the problem in domain-precise terms beyond what
a beginner would reach for. Merely naming files, paths, or visible APIs
does NOT count — those are available to anyone looking at the codebase.
Verification or correction is moderately specific — points to a particular
mechanic or asks for a specific piece of state ('did you actually...',
'show me the config for...') at least once. The user catches at least one
of Claude's domain mistakes or steers away from a wrong approach; Claude
does not have to correct the user's domain model.
- 5: Expert. Framing or steering shows ANY of: insider-only naming
(jargon / conventions / library internals that only a practitioner uses),
unprompted discussion of tradeoffs, surgical verification asks ('what's
the blocking factor on dim 2 of that kernel?', 'is the policy evaluating
principal.group or request context?'), authoritative corrections invoking
specific technical reasoning, preemptive edge-case handling, or test
designs targeting specific failure modes. One or two of these in a
session is enough — do not require all of them. Direction of correction
is user→Claude, never Claude→user on domain matters. The defining vibe is
'insider talking to an equal' even when the session is short.
- Unclear: Reserve for sessions where the user's contribution is so thin
that NO direction-of-expertise signal exists — almost entirely tool calls
/ retry requests with no substantive framing at all. When ANY framing is
present, even sparse, pick the best-fit 1-5 level rather than abstaining.
```

Occupation (user profile)

This classifier guesses the user's likely occupation given the available signals in the session, such as hints from claude.md files and project memory, specific technical or industry jargon, and referenced artifacts. It chooses one of 23 major groups of the BLS Standard Occupational Classification, or, if there are not enough signals to make a solid guess, the classifier returns "Unclear."

Pick the SOC (Standard Occupational Classification, U.S. BLS) MAJOR GROUP that describes the USER'S own most likely real-world profession — NOT the occupation of whoever would typically perform THIS SESSION's work.

The distinction matters: a lawyer using Claude Code to build a weekend side project is, for THIS question, 'Legal Occupations' — the side signals (their language, their references, their files, their context) point to law, even if the work they are doing in this session is something else (e.g. Python coding).

What to look at:
- CLAUDE.md files the agent loads (any path). They often encode the user's professional context — 'this is a research repo for computational biology work', 'this repo is a trading strategy backtester', 'this is a law firm's document-automation tooling'. These are strong signals about the user's background.
- Memory files / directory names / repo structure referenced in the session. A user directory containing clinical_trial_loader.py, econ_analysis.py, or contract_review/ reveals the user's professional world.
- The user's vocabulary in their messages — technical shorthand, jargon only an insider would use, references to tools/frameworks/conventions from a specific field.
- Referenced artifacts — do they mention patents? legal filings? clinical data? financial reports? an instrument? a classroom? a shipping schedule? These leak profession.
- Domains mentioned in passing that the user doesn't explain — fluent name-dropping reveals professional background.

What NOT to use:
- Do NOT classify on the task being performed in this session. If the only signal is 'they are writing Python', that is not user-profile evidence, that is task evidence.
- Do NOT classify on the language/tools Claude Code itself uses. CC uses Bash/Read/Write for any user.

Use 'Unclear' when there is no affirmative side signal beyond the act of coding. The fact that someone is writing code in Claude Code is NOT evidence they are a software developer by profession — every CC user writes code. Only pick 'Computer and Mathematical' when there is a specific signal that software/data IS their profession: CLAUDE.md describing a software product or engineering team, professional dev vocabulary, references to deploy/CI/review process, etc. Otherwise, Unclear.

The options are below:
- Management Occupations: Business/organizational leadership, strategic planning, general operations management, HR leadership.
- Business and Financial Operations: Analysts, accountants, buyers, HR specialists, compliance, market research, fundraising.
- Computer and Mathematical: Software developers, data scientists, network architects, statisticians, researchers in computing or mathematics. Only pick this when there is an affirmative signal that software/data is the user's PROFESSION, not just what they are doing in this session.
- Architecture and Engineering: Engineers (hardware, civil, mechanical, electrical, chemical, industrial), architects, drafters.
- Life, Physical, and Social Science: Scientists in biology, chemistry, physics, economics, psychology; lab researchers.
- Community and Social Service: Counselors, social workers, clergy, community health workers.
- Legal Occupations: Lawyers, paralegals, judges, legal support.
- Educational Instruction and Library: Teachers, professors, tutors, librarians, instructional designers, curriculum developers.
- Arts, Design, Entertainment, Sports, and Media: Writers, editors, designers, artists, musicians, journalists, translators, technical writers.
- Healthcare Practitioners and Technical: Doctors, nurses, therapists, pharmacists, diagnostic technicians.
- Healthcare Support: Aides, orderlies, medical assistants.
- Protective Service: Police, firefighters, security guards, correctional officers.
- Food Preparation and Serving: Chefs, cooks, servers, bartenders.
- Building and Grounds Cleaning and Maintenance: Custodians, groundskeepers, pest control.
- Personal Care and Service: Hairstylists, childcare, fitness trainers, funeral services.
- Sales and Related: Sales reps, retail associates, real estate agents, insurance agents.
- Office and Administrative Support: Admin assistants, secretaries, billing clerks, receptionists, office operations.
- Farming, Fishing, and Forestry: Farmers, ranchers, agricultural workers, loggers.
- Construction and Extraction: Construction trades, carpenters, electricians, plumbers, miners.
- Installation, Maintenance, and Repair: Mechanics, technicians, repair of machinery or vehicles.
- Production: Manufacturing workers, assemblers, machinists, printers.
- Transportation and Material Moving: Drivers, pilots, logistics, warehouse workers.
- Military Specific: Military-only occupations (combat specialists,

```
special forces).
- Unclear: Use this when there is NO affirmative side signal about the
user's professional world beyond the act of coding itself. The fact that
the user is writing code in Claude Code is NOT evidence of profession —
every CC user writes code. If the only signals are 'writes Python',
'uses git', 'edits files', pick Unclear, do NOT default to Computer
and Mathematical.
```

Occupation (work performed)

This classifier categorizes the occupation that the work performed in the session is typically associated with, choosing one of 23 major groups of the BLS Standard Occupational Classification.

```
Pick the SOC (Standard Occupational Classification, U.S. BLS) MAJOR GROUP
whose workers would typically perform the kind of work the user is doing
in this Claude Code session. Match on the WORK ITSELF, not on the session's
topic — a session about drafting a contract might still be performed by a
paralegal, a product manager, or a writer; only pick 'Legal Occupations' if
the user is doing lawyer-style legal analysis. Pick EXACTLY ONE of the 23
SOC major groups — there is no Unclear option. If signal is thin, pick the
best-fit group from whatever the transcript shows.

The options are below:
- Management Occupations: Business/organizational leadership, strategic
planning, general operations management, HR leadership.
- Business and Financial Operations: Analysts, accountants, buyers, HR
specialists, compliance, market research, fundraising.
- Computer and Mathematical: Software developers, data scientists, network
architects, statisticians, researchers in computing or mathematics.
- Architecture and Engineering: Engineers (hardware, civil, mechanical,
electrical, chemical, industrial), architects, drafters.
- Life, Physical, and Social Science: Scientists in biology, chemistry,
physics, economics, psychology; lab researchers.
- Community and Social Service: Counselors, social workers, clergy,
community health workers.
- Legal Occupations: Lawyers, paralegals, judges, legal support — actual
```

```
legal analysis and drafting, not just touching a legal topic.
- Educational Instruction and Library: Teachers, professors, tutors,
librarians, instructional designers, curriculum developers.
- Arts, Design, Entertainment, Sports, and Media: Writers, editors,
designers, artists, musicians, journalists, translators, technical writers.
- Healthcare Practitioners and Technical: Doctors, nurses, therapists,
pharmacists, diagnostic technicians.
- Healthcare Support: Aides, orderlies, medical assistants.
- Protective Service: Police, firefighters, security guards,
correctional officers.
- Food Preparation and Serving: Chefs, cooks, servers, bartenders.
- Building and Grounds Cleaning and Maintenance: Custodians, groundskeepers,
pest control.
- Personal Care and Service: Hairstylists, childcare, fitness trainers,
funeral services.
- Sales and Related: Sales reps, retail associates, real estate agents,
insurance agents.
- Office and Administrative Support: Admin assistants, secretaries, billing
clerks, receptionists, office operations.
- Farming, Fishing, and Forestry: Farmers, ranchers, agricultural
workers, loggers.
- Construction and Extraction: Construction trades, carpenters,
electricians, plumbers, miners.
- Installation, Maintenance, and Repair: Mechanics, technicians, repair of
machinery or vehicles.
- Production: Manufacturing workers, assemblers, machinists, printers.
- Transportation and Material Moving: Drivers, pilots, logistics,
warehouse workers.
- Military Specific: Military-only occupations (combat specialists,
special forces).
```

Session outcome (judged success)

This classifier judges whether the user, overall, accomplished what they set out to do. It first identifies the user's primary objective in the session and then judges the outcome.

```
Judge the OUTCOME of this Claude Code session — did the user accomplish
what they set out to do?

Pick EXACTLY ONE of: Succeeded, Partially Succeeded, Failed, or No clear
goal. There is NO 'Cannot Judge' option — pick the best fit from the four,
even when evidence is thin.

This facet measures ONE THING: the outcome (succeeded / partially succeeded
/ failed / no clear goal). It does NOT measure how verifiable the outcome
was — verifiable wins (commits, test passes, explicit 'ship it') and
inferred wins (clean ending with no complaint) both belong in 'Succeeded'.
Verifiable failures (errors, explicit 'didn't work') and inferred failures
(looping agent, silent abandonment) both belong in 'Failed'.

This is a TWO-STEP judgment — do both steps before you output a label:
**Step 1 — Identify the user's PRIMARY OBJECTIVE.**
Read the whole transcript and infer what the user was trying to accomplish
overall. This is usually set by the first few user messages but may
evolve over the session. If the user had multiple sub-tasks, pick the most
significant one (or the only unifying one). Some sessions have no clear
objective — pure exploration, chat, open-ended 'what do you think' — in
which case the objective is genuinely absent and the answer is 'No clear
goal'.

Long sessions evolve through phases (plan → execute → verify). Identify
the dominant operational objective by what the bulk of the session WORK
serves, not by the first message's framing alone. If the user opens with
a planning question ('can you suggest a plan?', 'how would you approach
this?') BUT the session then pivots to executing on the planned work, the
goal IS the planned work — pick that. 'No clear goal' is reserved for
sessions that stay exploratory throughout, with no execution phase. If the
agent committed code, ran a successful build, or produced a final artifact,
the goal is whatever produced that artifact.

**Truncation note:** if the transcript shows a `[TRANSCRIPT TRUNCATED FOR
LENGTH]` marker, the cut is NOT outcome evidence — it just means there was
more content than fit. Default to the strongest signal you can see in the
visible head + tail. If neither side is visible, judge from the trajectory:
```

clean / agent-on-task → Succeeded; loop / error / complaint → Failed; mixed → Partially Succeeded.

Step 2 — Judge the OUTCOME.
Read the whole transcript and decide which label best describes what happened, using whichever signals are available — hard or soft.

Signals pointing toward SUCCEEDED (any of these — hard or soft):
- A git commit / PR open / PR merge that matches the objective
- A test suite passing on the change, or a command running successfully to completion producing the requested output
- Explicit user affirmation — 'thanks', 'ship it', 'perfect', 'this works', 'exactly right'
- The final artifact exists and the user doesn't complain
- The session ends cleanly with the agent having done what was asked, no errors, no complaints, no goal-reframing

Signals pointing toward FAILED (any of these — hard or soft):
- User explicitly says it didn't work / is wrong / nevermind / 'this is broken'
- Final tool calls error out and are not recovered
- Tests fail at the end
- The agent loops, gives up, or apologizes without fixing
- The user abandons mid-task in a way that signals the goal was not met

Pick the label that best fits:
- Succeeded — outcome signals point toward the goal being met. The modal short CC session that ends cleanly with the agent having done what was asked belongs here. Don't downgrade just because no commit or test confirmed it. When the transcript is thin and you cannot tell from explicit signals, default to Succeeded if the trajectory is clean and the agent is on-task.
- Partially Succeeded — some sub-tasks done, others not; user accepted reduced scope; outcome is genuinely mixed.
- Failed — outcome signals point toward the goal NOT being met, whether by explicit error/complaint or by quiet abandonment / looping / clearly-fell-short work. When the transcript is thin and you cannot tell from explicit signals, pick Failed only if the trajectory looks broken (errors, looping, complaints).
- No clear goal — no operational objective in the first place; success frame doesn't apply.

```
DO NOT confuse ‘the agent worked competently’ with ‘the user’s goal was
achieved’. The agent can write clean code that does the wrong thing. Judge
OUTCOME vs. goal, not agent quality.

The options are below:
- Succeeded: The user’s primary objective was accomplished. Use this
whenever the best read of the transcript is that the user got what they
wanted — whether that judgment rests on HARD signals (a git commit
matching the work, a test passing, a PR opened/merged, the user saying
‘thanks’ / ‘perfect’ / ‘ship it’, a command running to completion with
output matching the objective, an artifact the user confirms) or SOFTER
signals (the agent produced what was asked, the session ended cleanly, the
user did not push back). This bucket folds together the modal short CC
session that ended without complaint AND the longer session that ended with
explicit verification — both are ‘Succeeded’.
- Partially Succeeded: Some but not all of the primary objective was
achieved. The user accepted a reduced scope, moved goalposts mid-session,
acknowledged one piece worked and another didn’t, or the agent made
progress on parts of the task but hit a wall on a specific sub-task. Use
when the outcome is genuinely mixed — neither a clean success nor a clean
failure.
- Failed: The user’s primary objective was NOT achieved. Use this whenever
the best read of the transcript is that the user did not get what they
wanted — whether that judgment rests on HARD signals (final commands
errored out and weren’t recovered, tests failed at the end, the user
explicitly said ‘this didn’t work’ / ‘nevermind’ / expressed frustration)
or SOFTER signals (the agent looped or gave up, the work clearly fell short
of the ask, the user abandoned mid-task in a way that signals the goal was
not met). Like ‘Succeeded’, this folds verifiable and inferred failures
into one outcome bucket.
- No clear goal: The user did not have a single discernible primary
objective — exploratory chat, wandering conversation, open-ended ‘what do
you think’ with no operational outcome requested. Success is not the right
frame for this session. Use when the frame doesn’t apply.
```

Success signal

This classifier looks for signals of success across the session such as git activity, successful tests, and user affirmation. It is used in conjunction with the session outcome classifier throughout the report, e.g., in the definition of verified success. If there are no observable failure signals, it returns “None.”

Judge how STRONGLY the transcript verifies that success-shaped work happened in this Claude Code session — on a 1-5 ordinal scale, or NONE if there is no success-shaped signal at all. This measures evidence strength on the success side, regardless of whether the session actually succeeded.

The 1-5 scale (with NONE as the explicit zero):
• NONE — no observable success-shaped signal at all
• 1 — barely any signal; faint trace; nothing verifiable
• 2 — soft signal: clean ending, no complaint, no commit/test/explicit affirmation (the modal short CC session)
• 3 — moderate signal: clean ending + at least one concrete piece of corroborating evidence (successful command output, artifact whose content matches ask, neutral user acknowledgement)
• 4 — strong signal: at least one HARD verifiable signal (commit / test pass / command-completion-with-matching-output / explicit user affirmation / user-confirmed artifact)
• 5 — very strong signal: MULTIPLE corroborating hard signals

Pick the highest tier that the transcript supports. The scale is ordinal — moving up requires MORE evidence, not different evidence.

DO NOT confuse 'the agent worked competently' with 'success was verified'. The agent can write clean code that does the wrong thing — that is at most a 2 or 3. A high score here just means strong EVIDENCE on the success side.

The options are below:
- 1: Very weak. Barely any success-shaped signal — a stray polite remark, a minor 'ok', the agent producing something inconsequential. Nothing verifiable; nothing the user actively endorsed. Use when you can detect the faintest positive trace but don't want to call it 'NONE'.
- 2: Soft. Agent produced what was asked and the artifact exists at session end; session ended cleanly with no errors, no complaints, no goal-reframing; user did not push back. No commit / test / explicit affirmation. The MODAL pattern for many short CC sessions.
- 3: Moderate. Clean ending PLUS at least one concrete piece of corroborating evidence — a successful command running to completion, an artifact whose content visibly matches the ask, a neutral user acknowledgement that the work was received. Stronger than 'Soft' because there is concrete evidence the work hit the target, but no single HARD signal yet.
- 4: Strong. At least one HARD verifiable success signal: a git commit whose message matches the work, a PR opened or merged, a test suite passing on the change, a command running to completion with output matching the

```
stated objective, an explicit user affirmation (‘thanks’, ‘perfect’, ‘ship
it’, ‘this works’, ‘exactly right’, concrete praise), OR a final artifact
the user explicitly confirms. One hard signal, on its own.
- 5: Very strong. MULTIPLE corroborating hard signals — e.g., commit +
passing test, OR explicit ‘ship it’ + a confirmed artifact, OR PR merged
+ user thanks, OR test pass + explicit verification by the user. The
transcript leaves no room to doubt that the success happened.
- NONE: No observable success-shaped signal at all. The session may have
failed (errors, abandonment, explicit complaints), may have been cut off
before any work could complete, may be pure exploration / open-ended chat
where success isn’t the right frame, or may be so tool-noise-dominated that
no user-facing artifact is observable.
```

Failure signal

This classifier looks for signals of failure across the session such as user concern or frustration, failing tests, or looping. If there are no observable success signals, it returns “None.”

```
Judge how STRONGLY the transcript verifies that failure-shaped work
happened in this Claude Code session — on a 1-5 ordinal scale, or NONE if
there is no failure-shaped signal at all. This measures evidence strength
on the failure side, regardless of whether the session actually failed.

The 1-5 scale (with NONE as the explicit zero):
• NONE — no observable failure-shaped signal at all
• 1 — barely any signal; faint trace; immediately recovered
• 2 — soft signal: agent looped briefly, work mildly off-target, user
expressed mild concern
• 3 — moderate signal: agent gave up on a sub-task, work clearly fell short
on one dimension, user pushed back without explicit rejection
• 4 — strong signal: at least one HARD verifiable signal (unrecovered final
error, failing test at end, explicit user complaint or rejection, expressed
frustration)
• 5 — very strong signal: MULTIPLE hard failure signals
```

```
Pick the highest tier that the transcript supports. The scale is ordinal — moving up requires MORE evidence, not different evidence.

Two traps to avoid: (1) a single intermediate tool error that the agent recovers from is at most a 1 — the FINAL state is what counts. (2) Polite user disengagement at the end of a clean session is NOT a failure signal — silence after success-shaped work is just silence.

The options are below:
- 1: Very weak. Barely any failure-shaped signal — a fleeting agent apology that was immediately recovered, a transient retry, a minor user nitpick. Use when you can detect the faintest negative trace but don't want to call it 'NONE'.
- 2: Soft. Agent looped briefly, OR work is mildly off-target, OR user expressed mild concern that the agent addressed. The session reads as imperfect but not failure-shaped overall.
- 3: Moderate. Agent gave up on a sub-task, OR work clearly fell short on one dimension, OR user pushed back without explicitly rejecting the work. There is concrete evidence of friction but no hard final failure.
- 4: Strong. At least one HARD verifiable failure signal: final tool calls error out and are not recovered, a test suite fails at the end of the session, the user explicitly says it didn't work / 'this is broken' / 'nevermind' / expresses concrete frustration, or the user explicitly rejects the work the agent produced. One hard signal, on its own.
- 5: Very strong. MULTIPLE hard failure signals — errors + explicit complaint + abandonment, OR multiple failed sub-tasks without recovery, OR the user repeatedly rejecting work. The transcript leaves no room to doubt that the failure happened.
- NONE: No observable failure-shaped signal. The session may have succeeded (cleanly or verifiably), may be open-ended chat where failure isn't the right frame, or may be cut off before any outcome could form.
```

Estimated task value (freelance-equivalent price)

The task-value estimate is produced in two model calls. The first call (Claude Haiku 4.5) rewrites the session as a freelance job posting:

```
Read this Claude Code session transcript and describe it as an equivalent
Upwork-style job posting — as if a client were hiring a freelancer to
perform the work Claude actually did in the session.

Write as the client would, not the freelancer. Focus on scope and
deliverables. Do not mention Claude, AI, or that this came from a Claude
Code session.

Output a single JSON object only, no commentary:
{
  "title": "short listing-style title, 80 chars or less",
  "description": "3-5 sentences describing the work and deliverables",
  "skills": ["3-8 skill tags"]
}
<transcript>
{TRANSCRIPT}
</transcript>
```

The second call (Opus 4.7) prices that posting against a calibration corpus of 200 real freelance postings with their actual prices—a tier-balanced sample drawn from a public job-postings datasets from 2024-2026. Postings are eligible for the calibration sample if they are fixed-price, priced between $10 and $50,000, and have a description of at least 50 characters. About 23,000 postings survive these filters.

Below, we discuss the development and validation of this classifier in more detail.

Validation on a stratified holdout

In a holdout study with 999 job postings, the task value estimator reaches log $R^2 = 0.38$. To construct the holdout, we gathered 250 examples from four price tiers ($10–200, $200–1,000, $1,000–5,000, and $5,000–50,000), drawn from the same filtered corpus described above.

We report three metrics: log R^2 (Pearson R^2 between log actual and log predicted price), level R^2 (the same in raw dollars), and multiplicative bias (the geometric mean of predicted/actual). Confidence intervals are 1,000-draw bootstraps.

Tier	n	log R^2 [95% CI]	level R^2 [95% CI]	Bias	Median pred.	Median actual
Overall	999	0.38 [0.32, 0.44]	0.38 [0.32, 0.44]	0.66	$500	$900
$10–200	250	0.16 [0.08, 0.25]	0.16 [0.08, 0.25]	2.39	$150	$50
$200–1,000	250	0.07 [0.02, 0.14]	0.07 [0.02, 0.14]	1.10	$400	$300
$1,000–5,000	249	0.06 [0.02, 0.13]	0.06 [0.02, 0.13]	0.53	$1,000	$1,500
$5,000–50,000	250	.01 [0.00, 0.03]	.01 [0.00, 0.03]	0.14	$2,000	$8,000

Note that the overall 0.38 largely reflects cross-tier ordering—because the holdout weights all four tiers equally, the pricer earns credit for placing $50 jobs below $8,000 jobs even where it cannot rank jobs within a tier. Within-tier discrimination is concentrated at the low end and is essentially zero above $5,000.

Note also that bias reverses across the range. Small jobs are over-priced about 2.4x, the largest under-priced about 7x. So, we generally use these task value estimates ordinally rather than using them to get aggregate task values.

Choice of calibration examples

For calibration, we use the same tiers used in validation ($10–200, $200–1,000, $1,000–5,000, and $5,000–50,000). The filtered corpus of job postings is heavily skewed toward small jobs (the distribution across the four tiers is roughly 62% / 27% / 9% / 2%).

We tested multiple options for the calibration set. With a uniformly weighted draw, the pricer anchors to the small-job range and compresses everything above it: on the validation holdout it under-priced $5,000+ jobs by roughly 12x (median prediction $800).

Balancing the calibration set to 50 examples per tier raised the prediction level for larger jobs (median prediction for $5,000+ jobs rose to $2,000) and gave the best overall accuracy. On small jobs, ranking accuracy was statistically unchanged, at the cost of somewhat more over-pricing. Since the report uses

the estimates comparatively rather than as dollar levels, we took the variant with the best overall ranking and the least price-dependent distortion.

Choosing the pricing model

Holdout accuracy for this task is model-dependent, so the cited validation numbers are specific to the exact model and prompt the pipeline runs. We ran a grid of two models (Sonnet 4.6 and Opus 4.7) and two calibration variants (uniform draw from freelance corpus and tier-balanced) on the stratified holdout:

Pricing model	Calibration examples	Log R^2	Level R^2	Bias (geomean)
Sonnet 4.6	uniform	0.33	0.11	0.56
Sonnet 4.6	tier-balanced	0.33	0.09	0.86
Opus 4.7	uniform	0.36	0.06	0.46
Opus 4.7	**tier-balanced**	**0.38**	**0.09**	**0.66**

Derived measures and definitions

Verified success. A session reaches verified success when the outcome classifier judges it Succeeded AND the success-signal classifier scores 4 or 5—i.e., the transcript carries at least one hard verifiable signal (a matching commit, a passing test suite, a command completing with output matching the objective, or explicit user confirmation).

Hits trouble. A session hits trouble when the failure-signal classifier scores 3 or higher: concrete evidence of struggle—an error, a failed test, an abandoned sub-task, repeated attempts, or the user pushing back.

Abandoned. A troubled session is abandoned when the outcome classifier judges it Failed AND telemetry records zero lines of code added.

Wrote code. A session wrote code when telemetry records at least one line of code added.

Software/math vs. other professions. A user is counted in software-related occupations when the user-profile occupation classifier returns Computer and Mathematical; "other identified professions" are all other SOC major groups. Sessions whose user profile is Unclear are excluded from occupation analyses (about 30% of sessions).

Classifier validation and cross-check

Classifiers versus telemetry

Outcome-classifier. The success-signal and failure-signal classifiers track the independent outcome judgment in the expected directions: the share of sessions judged Succeeded rises from 13% at success-signal 1 to 90% at success-signal 5, and falls from 85% at failure-signal 1 to 3% at failure-signal 5 (Figure A1).

Decision attribution. Sessions the classifier scores as more Claude-led on planning show more model calls per human prompt and more tool calls per

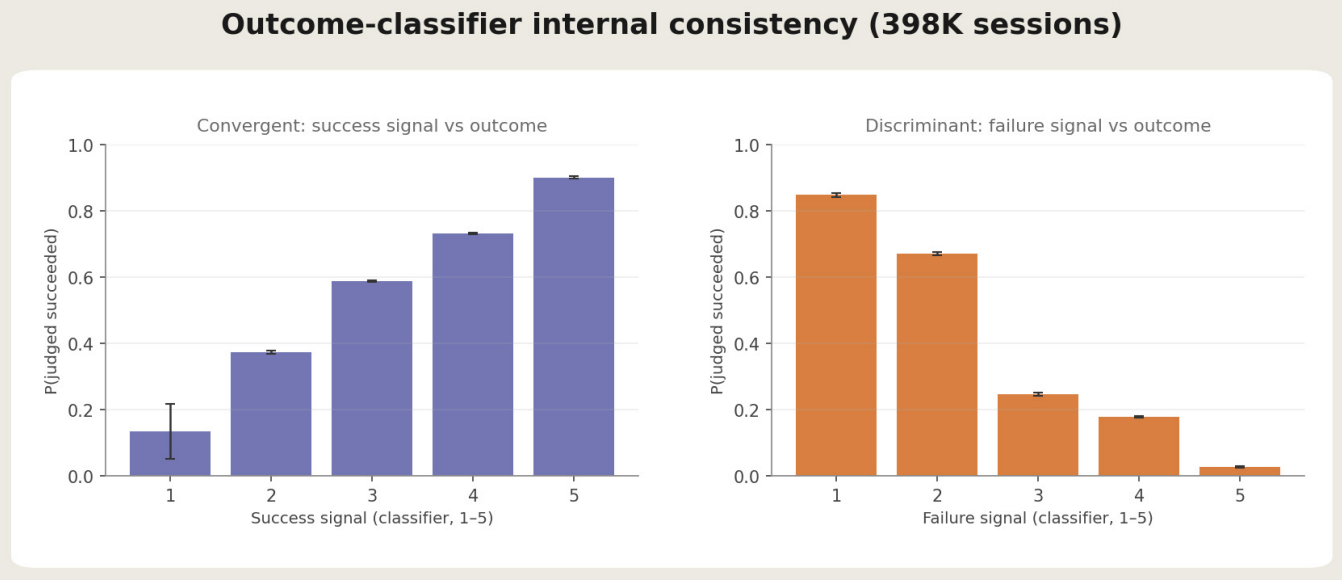

Figure A1. Success outcome versus success and failure signals
The left panel shows the probability that the success outcome classifier records "succeeded" for sessions of each success signal strength. The right panel shows the probability that the success outcome classifier records "succeeded" for sessions of each failure signal strength.

human prompt, convergence between a transcript-based judgment and harness-recorded telemetry. Execution share is near-orthogonal to these (Figure A2).

Internal outcome validation

We ran the same classifier prompts on internal Anthropic Claude Code sessions, where session IDs can be joined to observable engineering outcomes (whether the session's commits landed on the main branch). The expertise–success gradient replicates on this sample, including within-engineer (comparing sessions of the same person at different rated expertise levels): judged success rises by roughly 3–5 percentage points per expertise level with author fixed effects, and sessions at higher rated expertise are more likely to produce code that lands.

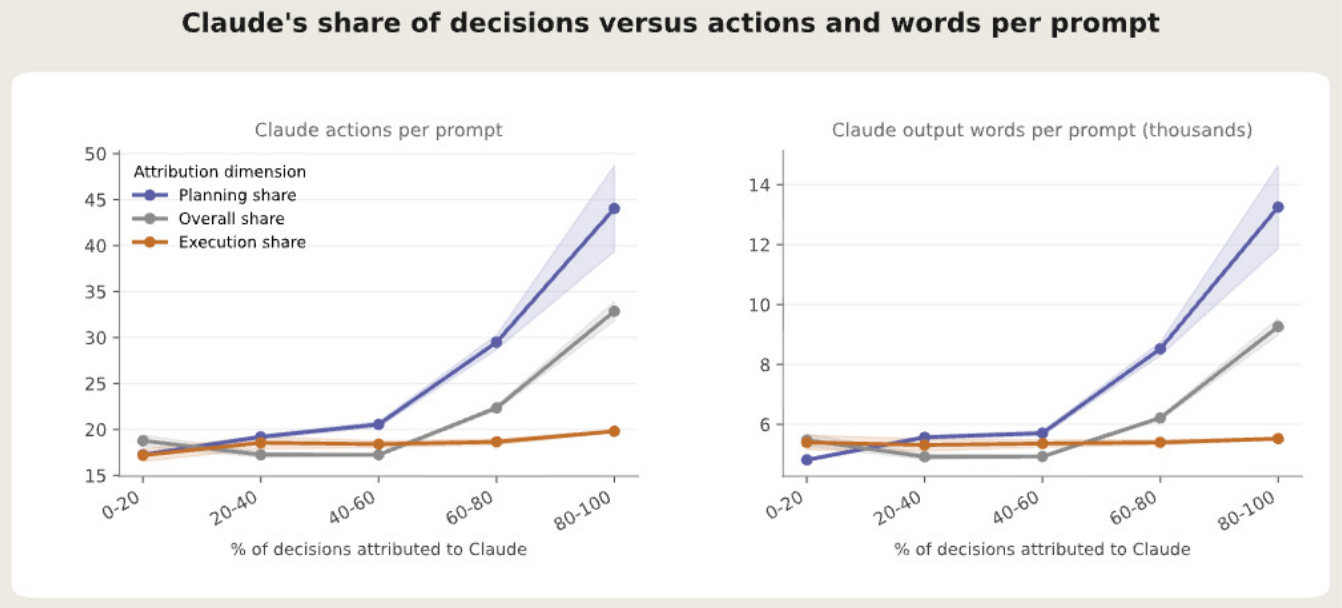

Figure A2. Claude's share of decisions versus actions The left panel shows Claude's average actions per prompt for each band of decision share attributed to Claude. The right panel shows Claude's average words per prompt for each decision share band.

Classifier validation via strong-model agreement

It is challenging to validate classifiers on Claude Code sessions—a single session can span hundreds of turns, files, and tool calls, and hours (or even days) of work. The norm for classifier validation (pursued in previous reports) has been to benchmark the classifier's performance against human performance for a subset of session transcripts that are accessible from a

privacy perspective. But the length and complexity of Claude Code sessions call into question whether such an approach is feasible and reliable. It is not clear that humans, with their limited working memory and limited understanding of unfamiliar codebases and agentic tool traces, can label sessions more consistently than a powerful model.

For this report, we experimented with a different approach to validation. We re-ran the report's classifiers (using the models used in the report) on 198 sessions from SWE-chat, a public dataset of real coding-agent sessions recorded from public GitHub repositories across several harnesses. We then used a strong reference model (Mythos Preview) to label the same sessions, treating the reference model labels as reliable and consistent proxies for ground-truth. We computed the disagreements between the reference model and the models used in the report. Then, a blind judge (again Mythos Preview) adjudicated every disagreement, noting whether the disagreement was due to genuine ambiguity (it could go either way) or represented a clear failure on the part of the report's classifier. We brought a human into the loop for one final step—one of the authors of this report blindly re-adjudicated all concerning disagreements.

The report's classifier and the reference agree on 78-98% of sessions on the categorical classifiers. On the classifiers with ordinal scales, the report's classifier and the reference agree on 78-99% counting adjacent scale points as agreement, and 53-68% counting only exact matches as agreement. The judge rated a vast majority of the disagreements as genuinely ambiguous—concerning disagreements are 0-3% of sessions per classifier (at most 5 of 198). In the final step, a human reviewer audited the 15 disagreements the judge had flagged as concerning and concurred with the judge in 11 of 15 sessions.

Taken together, the results of our validation are encouraging. They do caution against over-reading any particular label, but most of the report's conclusions rest on relative comparisons, i.e., differences between groups and changes over time, and on those, the classifier is more reliable. As long as the classifier's mistakes are similar across groups (which we did not extensively verify here), those mistakes cancel each other out when making comparisons.

This validation approach is far from perfect, but we do think it is promising to use a strong reference model alongside blind adjudication of disagreements,

Classifier	Type	Exact agreement	Within 1-point agreement	Concerning disagreements
Session success	Categorical	82%	—	3 (2%)
Work mode	Categorical	78%	—	5 (3%)
Occupation (work)	Categorical	98%	—	1 (0.5%)
Occupation (user)	Categorical	96%	—	2 (1%)
User expertise	Ordinal	68%	99%	0
Failure signal	Ordinal	64%	85%	2 (1%)
Success signal	Ordinal	53%	78%	2 (1%)

with human review reserved for spot-checking the judge's calls (less as ground truth and more as a way for the authors to build intuition for what the disagreements actually look like). We are actively developing methods of validation that take seriously that human labels may no longer be the gold standard for complex sessions and also that some human review may still be valuable for interpretation. We encourage others in the research community to take up this methodological challenge alongside us.

Robustness of the expertise-success gradient

This section turns to the central claim of Section 4—that a session's chance of reaching verified success rises with the user's rated expertise at the task. We show regressions underlying the claims.

Verified success and expertise under cumulative controls

Each column in the table below adds a control, testing a different hypothesis. First, that expert-rated users pick different kinds of work (column 2), work in different months (3), work on different subjects (4), hold different occupations

	(1)	(2)	(3)	(4)	(5)	(6)
Expertise (per level, 1–5)	+0.0426	+0.0507	+0.0443	+0.0442	+0.0379	+0.0364
	(0.0008)	(0.0008)	(0.0008)	(0.0008)	(0.0009)	(0.0009)
Work mode × task-value band FE		✓	✓	✓	✓	✓
Month FE			✓	✓	✓	✓
Task-occupation FE				✓	✓	✓
User-occupation FE					✓	✓
Model-tier × session-length-band FE						✓
ln(model calls), ln(human turns)						✓
Sessions	359,586	359,586	359,586	353,347	351,603	302,647
Fixed-effect cells	—	36	252	615	1,130	4,438
R^2	0.009	0.066	0.081	0.083	0.088	0.136

Table A1. Verified success and expertise under controls
OLS with fixed effects. 375,687 sessions, columns (4)-(6) have fewer due to cells that fell under the aggregation limit. Standard errors in parentheses, clustered by user. Column (5) is the preferred specification. It compares sessions doing the same kind of work, at the same estimated value, in the same month, on the same type of task, from users in the same occupational group. Column (6) additionally conditions on choices made during the session (model, length). Excludes sessions with no clear goal.

(5), or run longer sessions on stronger models (6). The coefficient is the change in the probability of verified success per step on the five-point expertise scale.

Verified success by expertise level (step function)

Table A1 assumes that expertise is a linear scale, i.e., that each step of the expertise scale is worth the same. Table A2 instead uses an indicator for each level of the expertise scale, under the Table A1 column-(5) fixed effects, with Novice as the omitted category. It is the construction behind Figure 6.

Expertise	Sessions	Unadjusted rate	Coefficient (vs. Novice)	Adjusted rate
1 (Novice)	5,911	13.2%	—	14.5%
2 (Beginner)	73,858	20.9%	+6.4 (0.7)	20.9%
3 (Intermediate)	96,516	28.0%	+13.8 (0.7)	28.3%
4 (Advanced)	146,879	29.3%	+15.0 (0.7)	29.5%
5 (Expert)	36,422	34.7%	+18.4 (0.7)	32.9%

Table A2: Verified success versus expertise, as a step function
OLS with the Column (5) fixed effects from Table A1. Standard errors in parentheses, clustered at the user level. Novice is the reference category, so its coefficient is not defined. The unadjusted rate is the raw share of sessions at each expertise level ending in verified success, with no controls. The adjusted rate holds the fixed effects constant. Excludes sessions with no clear goal.